

INFORME DE TALLER PRÁCTICO

Terraform + Docker

Automatizando una arquitectura Node.js + React + Nginx + MySQL, localmente con Docker

Jairo Miguel Lara Serrano

Jumandi Andrade

Klever Guamushig

Guillermo Vallejo

Maestría en Ingeniería de Software

Desarrollo de Aplicaciones Empresariales

29 de agosto de 2026

<https://github.com/miguesj-hub/taller-terraform-docker>

Índice

1. Objetivo del taller	2
2. Arquitectura desplegada	2
3. Entorno y estructura del proyecto	3
4. Ciclo básico: de <code>init</code> a la aplicación funcionando	3
4.1. Paso 1 — Inicializar	3
4.2. Paso 2 — Formatear y validar	4
4.3. Paso 3 — Planificar	4
4.4. Paso 4 — Aplicar	5
4.5. Paso 5 — Verificar	7
5. El archivo de estado	8
5.1. Inspeccionar el estado	8
5.2. Idempotencia	11
5.3. Provocar y detectar <i>drift</i>	12
5.4. Corregir el <i>drift</i>	13
5.5. Reemplazar un recurso puntual	15
5.6. Renombrar una dirección sin destruir el recurso	16
5.7. Importar un recurso creado manualmente	17
5.8. Limpieza del ejercicio y una observación	19
5.9. Resumen de la comparación con Docker Compose	20
6. Escalar el backend	20
7. Destruir el entorno	23
8. Conclusiones	24

1. Objetivo del taller

El taller consiste en desplegar, con Terraform y sin ningún proveedor de nube, una arquitectura completa de cuatro capas sobre contenedores Docker locales, y usarla como banco de pruebas para practicar el archivo de estado (`terraform.tfstate`).

Los objetivos concretos son cinco:

1. Desplegar la arquitectura Nginx (balanceador) → React (frontend) → Node.js (backend, con réplicas) → MySQL, usando el proveedor `docker` de Terraform.
2. Ejecutar el flujo completo `init` → `plan` → `apply` → `destroy`.
3. Practicar de forma guiada el archivo de estado: inspeccionarlo, provocar y detectar *drift*, reemplazar recursos puntuales, renombrar direcciones sin destruir infraestructura e importar un recurso creado a mano.
4. Escalar el backend cambiando una sola variable y observar cómo Terraform recalcula el plan y regenera la configuración de Nginx.
5. Revisar qué haría falta antes de llevar este mismo patrón a producción.

Todo corre en local: no se usó AWS, GCP ni Azure en ningún momento, y el taller no consumió crédito de nube.

El código completo del taller, junto con este informe, está disponible en el repositorio <https://github.com/miguesj-hub/taller-terraform-docker>.

2. Arquitectura desplegada

Nginx es el único servicio con un puerto publicado al host (8080). Reparte `/api/*` entre las réplicas del backend mediante balanceo *round-robin* y sirve `/` haciendo proxy al contenedor del frontend. Todo lo demás vive dentro de la red Docker privada `iac-workshop-net`, sin exposición al exterior.

Tabla 1: Componentes de la arquitectura

Componente	Función	Puerto
Nginx	Balanceador y <i>reverse proxy</i> ; único punto de entrada	8080 (host)
React (frontend)	Interfaz servida como estáticos desde su propio Nginx interno	80 (interno)
Node.js/Express	API con <code>/api/health</code> y <code>/api/visits</code> ; N réplicas	3000 (interno)
MySQL 8.0	Base de datos con volumen nombrado para persistencia	3306 (interno)

Dos detalles del diseño que resultan importantes más adelante:

- El backend reporta su `hostname` en `/api/health`. Como Terraform fija ese `hostname` explícitamente (`backend-1`, `backend-2...`), el balanceo de carga se puede comprobar visualmente desde el navegador.
- La configuración de Nginx no es un fichero estático: se genera en el momento del `apply` con `templatefile()` a partir de la lista de réplicas. Cambiar el número de réplicas cambia ese contenido y obliga a recrear el contenedor de Nginx.

3. Entorno y estructura del proyecto

Tabla 2: Herramientas utilizadas

Herramienta	Versión mínima	Verificación
Docker Desktop / Engine	24.x	<code>docker version</code>
Terraform CLI	1.5.0	<code>terraform -version</code>
Node.js (opcional)	20.x	<code>node -v</code>

El proveedor `kreuzwerker/docker` habla directamente con el socket local de Docker, el mismo que usa el comando `docker`. Por eso no hacen falta credenciales de ningún tipo. El único acceso a internet que se necesita es el de la primera ejecución, para descargar el plugin del proveedor y las imágenes base.

Solo el directorio `terraform/` contiene código de infraestructura; `backend/`, `frontend/`, `nginx/` y `mysql/` son las aplicaciones y ficheros de configuración que Terraform construye y despliega.

```
workshop/
  backend/      API Express (health + visits)
  frontend/     UI React que consume la API
  nginx/        plantilla nginx.conf.tpl que Terraform renderiza
  mysql/        esquema inicial de la base de datos
  terraform/
    versions.tf  provider + version de Terraform
    variables.tf replicas, puerto, credenciales
    locals.tf    lista de alias backend-1, backend-2, ...
    network.tf   red privada iac-workshop-net
    mysql.tf     imagen + volumen + contenedor
    backend.tf   usa "count" para las replicas
    frontend.tf  una sola replica
    nginx.tf     usa templatefile() + upload
    outputs.tf   URLs y nombres de contenedores
```

4. Ciclo básico: de init a la aplicación funcionando

Todos los comandos se ejecutan dentro de `terraform/`.

4.1 Paso 1 — Inicializar

```
cd workshop/terraform
terraform init
```

Terraform descargó el plugin `kreuzwerker/docker` v4.5.0, preparó el directorio `.terraform/` y creó el fichero de bloqueo `.terraform.lock.hcl`, que fija esa versión exacta para futuras ejecuciones.

```
● mike@Miguels-MacBook-Pro workshop % cd terraform
● mike@Miguels-MacBook-Pro terraform % terraform init
  Initializing the backend...

  Initializing provider plugins...
  - Finding kreuzwerker/docker versions matching "~> 4.0"...
  - Installing kreuzwerker/docker v4.5.0...
  - Installed kreuzwerker/docker v4.5.0 (self-signed, key ID 0DCE698927DAF8EC)
  Partner and community providers are signed by their developers.
  If you'd like to know more about provider signing, you can read about it here:
  https://developer.hashicorp.com/terraform/cli/plugins/signing

  Terraform has created a lock file .terraform.lock.hcl to record the provider
  selections it made above. Include this file in your version control repository
  so that Terraform can guarantee to make the same selections by default when
  you run "terraform init" in the future.

  Terraform has been successfully initialized!

  You may now begin working with Terraform. Try running "terraform plan" to see
  any changes that are required for your infrastructure. All Terraform commands
  should now work.

  If you ever set or change modules or backend configuration for Terraform,
  rerun this command to reinitialize your working directory. If you forget, other
  commands will detect it and remind you to do so if necessary.
○ mike@Miguels-MacBook-Pro terraform %
```

Figura 1: terraform init: descarga del proveedor y creación del fichero de bloqueo de versiones.

4.2 Paso 2 — Formatear y validar

```
terraform fmt
terraform validate
```

`fmt` normaliza el estilo del código y `validate` revisa la sintaxis y las referencias internas. Ninguno de los dos necesita que Docker esté corriendo, así que sirven como comprobación barata antes de tocar nada real.

```
● mike@Miguels-MacBook-Pro terraform % terraform fmt
● mike@Miguels-MacBook-Pro terraform % terraform validate
  Success! The configuration is valid.

○ mike@Miguels-MacBook-Pro terraform %
```

Figura 2: La configuración es sintácticamente válida.

4.3 Paso 3 — Planificar

```
terraform plan
```

El plan anunció **11 recursos a crear** y ningún cambio ni destrucción: la red, el volumen de MySQL, cuatro imágenes y cinco contenedores. Todavía no se creó nada; este es el momento de leer con calma qué va a pasar.

```
Plan: 11 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ api_health_url      = "http://localhost:8080/api/health"
+ app_url             = "http://localhost:8080"
+ backend_container_names = [
+   "workshop-backend-1",
+   "workshop-backend-2",
+ ]
+ mysql_container_name = "workshop-mysql"
+ mysql_volume_name    = "iac-workshop-mysql-data"

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take
exactly these actions if you run "terraform apply" now.
mike@miguels-MacBook-Pro terraform %
```

Figura 3: Resumen del plan: 11 to add, 0 to change, 0 to destroy, junto con los valores que tomarán los *outputs*.

4.4 Paso 4 — Aplicar

```
terraform apply
```

```
Plan: 11 to add, 0 to change, 0 to destroy.

Changes to Outputs:
+ api_health_url      = "http://localhost:8080/api/health"
+ app_url             = "http://localhost:8080"
+ backend_container_names = [
+   "workshop-backend-1",
+   "workshop-backend-2",
+ ]
+ mysql_container_name = "workshop-mysql"
+ mysql_volume_name    = "iac-workshop-mysql-data"

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes
```

Figura 4: Terraform pide confirmación explícita antes de ejecutar el plan; solo se acepta *yes*.

Durante el `apply` se nota que MySQL tarda alrededor de un minuto en completarse: no basta con que el proceso arranque, porque el recurso declara un `healthcheck` con `wait = true` y Terraform no lo da por aplicado hasta que MySQL responde *healthy*. El `depends_on` del backend obliga a esperar a que eso ocurra.

```

Enter a value: yes

docker_image.mysql: Creating...
docker_image.nginx: Creating...
docker_network.workshop_net: Creating...
docker_volume.mysql_data: Creating...
docker_image.backend: Creating...
docker_image.frontend: Creating...
docker_image.nginx: Creation complete after 0s [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_volume.mysql_data: Creation complete after 2s [id=iac-workshop-mysql-data]
docker_network.workshop_net: Creation complete after 4s [id=bdd63b681a36238807005ffbe48ed9a6f3ab23f98553437df1e930da206f3ff7]
docker_image.frontend: Still creating... [00m10s elapsed]
docker_image.backend: Still creating... [00m10s elapsed]
docker_image.mysql: Still creating... [00m10s elapsed]
docker_image.backend: Creation complete after 19s [id=sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_image.mysql: Still creating... [00m20s elapsed]
docker_image.frontend: Still creating... [00m20s elapsed]
docker_image.mysql: Still creating... [00m30s elapsed]
docker_image.frontend: Still creating... [00m30s elapsed]
docker_image.frontend: Still creating... [00m40s elapsed]
docker_image.mysql: Still creating... [00m40s elapsed]
docker_image.frontend: Creation complete after 47s [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_container.frontend: Creating...
docker_container.frontend: Creation complete after 0s [id=736eedc20721f6f29aec40649a070b9e6195ba35646fd6fb43876854f2304e6d]
docker_image.mysql: Still creating... [00m50s elapsed]
docker_image.mysql: Creation complete after 54s [id=sha256:7dcdcc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_container.mysql: Creating...
docker_container.mysql: Still creating... [00m10s elapsed]
docker_container.mysql: Still creating... [00m20s elapsed]
docker_container.mysql: Still creating... [00m30s elapsed]
docker_container.mysql: Still creating... [00m40s elapsed]
docker_container.mysql: Still creating... [00m50s elapsed]
docker_container.mysql: Still creating... [01m00s elapsed]
docker_container.mysql: Creation complete after 1m5s [id=81fde5253f988e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.backend[1]: Creating...
docker_container.backend[0]: Creating...
docker_container.backend[0]: Creation complete after 1s [id=170d7ff57cddea7a2bb8db75229f14b5f2ebc3eb8a252211240fcdc4d79d69]
docker_container.backend[1]: Creation complete after 1s [id=f9e1e828a2558cbale36e82bb9fd1c57140f458a5c677bdd95583ac0b8de8d26]
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 0s [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]

Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

Outputs:

api_health_url = "http://localhost:8080/api/health"
app_url = "http://localhost:8080"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name = "iac-workshop-mysql-data"
○ mike@Miguels-MacBook-Pro terraform %

```

Figura 5: Creación de los 11 recursos en orden de dependencias. MySQL tarda 1m5s por el *healthcheck*; los backends se crean después, en paralelo.

```

Apply complete! Resources: 11 added, 0 changed, 0 destroyed.

Outputs:

api_health_url = "http://localhost:8080/api/health"
app_url = "http://localhost:8080"
backend_container_names = [
    "workshop-backend-1",
    "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform %

```

Figura 6: *Outputs* finales: URL de la aplicación, URL de salud de la API y nombres de los contenedores generados.

4.5 Paso 5 — Verificar

```

docker ps
curl http://localhost:8080/api/health
curl http://localhost:8080/api/visits

```

```

mike@Miguels-MacBook-Pro terraform % docker ps
CONTAINER ID   IMAGE          COMMAND                  CREATED        STATUS        PORTS                               NAMES
e628e258f1d7   65645c7bb6a0   "/docker-entrypoint.    About a minute ago    Up About a minute    0.0.0.0:8080->80/tcp    workshop-nginx
170d7ff57cdd   8a48594775c4   "/docker-entrypoint.s   About a minute ago    Up About a minute    3000/tcp                workshop-backend-1
f9e1e828a255   8a48594775c4   "/docker-entrypoint.s   About a minute ago    Up About a minute    3000/tcp                workshop-backend-2
81fde5253f98   7dcddc01f13b   "/docker-entrypoint.s   3 minutes ago        Up 2 minutes (healthy) 3306/tcp, 33060/tcp     workshop-mysql
736eedc20721   717da3a9b0ef   "/docker-entrypoint.    3 minutes ago        Up 3 minutes          80/tcp                  workshop-frontend

```

Figura 7: Los cinco contenedores en ejecución. Solo `workshop-nginx` publica un puerto al host (0.0.0.0:8080->80/tcp); MySQL aparece como *healthy*.

Abriendo `http://localhost:8080` se ve la interfaz de React leyendo el contador de visitas desde MySQL. El botón que dispara seis peticiones a `/api/health` muestra el balanceo en vivo: las respuestas se reparten entre `backend-1` y `backend-2`.

Taller IaC · Terraform + Docker

React → Nginx (balanceador) → Node.js (réplicas) → MySQL, todo desplegado con Terraform.

Conexión a la base de datos

Total de visitas registradas en MySQL: 1
Petición atendida por: backend-1

Balanceo de carga entre réplicas

Enviar 6 peticiones a /api/health

- Petición 1 → atendida por backend-2
- Petición 2 → atendida por backend-1
- Petición 3 → atendida por backend-2
- Petición 4 → atendida por backend-1
- Petición 5 → atendida por backend-2
- Petición 6 → atendida por backend-1

Figura 8: Aplicación funcionando con dos réplicas. Las seis peticiones alternan entre backend-2 y backend-1.

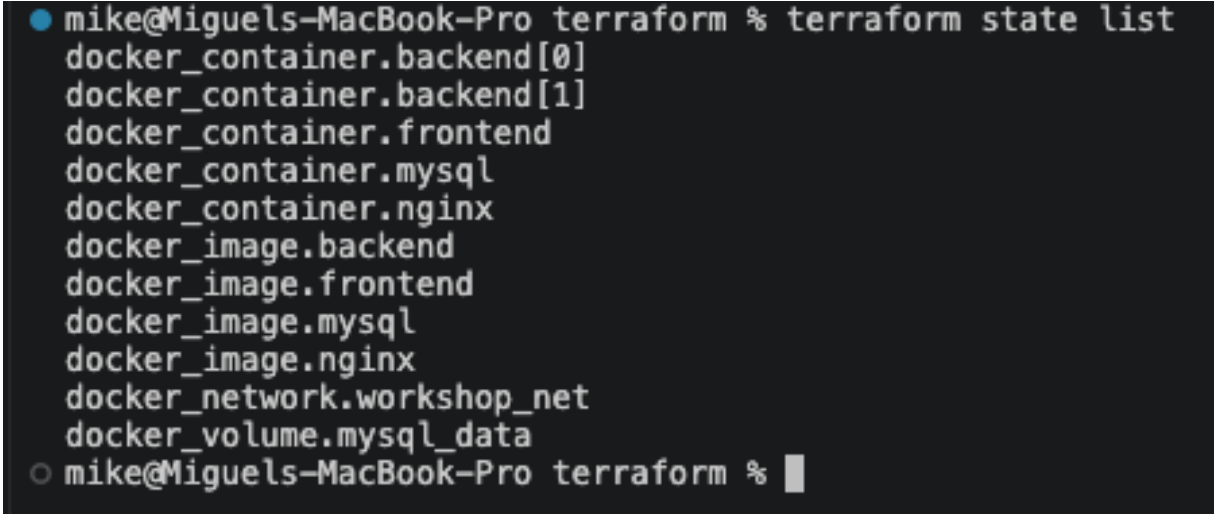
5. El archivo de estado

Esta es la parte central del taller y la que no tiene equivalente en Docker Compose. Compose puede levantar los mismos servicios, pero no mantiene un registro persistente y consultable de qué es cada recurso, con qué atributos y cómo se relaciona con los demás.

5.1 Inspeccionar el estado

```
terraform state list
```

Lista las direcciones de todos los recursos administrados, incluyendo el índice de cada réplica del backend.

A terminal window with a dark background and light-colored text. The prompt is 'mike@Miguels-MacBook-Pro terraform %'. The command 'terraform state list' has been executed, resulting in a list of 11 resource addresses. The first two are 'docker_container.backend[0]' and 'docker_container.backend[1]', which are highlighted with blue and orange circles respectively. The remaining resources are 'docker_container.frontend', 'docker_container.mysql', 'docker_container.nginx', 'docker_image.backend', 'docker_image.frontend', 'docker_image.mysql', 'docker_image.nginx', 'docker_network.workshop_net', and 'docker_volume.mysql_data'. The prompt 'mike@Miguels-MacBook-Pro terraform %' is followed by a cursor.

```
● mike@Miguels-MacBook-Pro terraform % terraform state list
docker_container.backend[0]
docker_container.backend[1]
docker_container.frontend
docker_container.mysql
docker_container.nginx
docker_image.backend
docker_image.frontend
docker_image.mysql
docker_image.nginx
docker_network.workshop_net
docker_volume.mysql_data
○ mike@Miguels-MacBook-Pro terraform %
```

Figura 9: Las once direcciones del estado. Nótese `docker_container.backend[0]` y `docker_container.backend[1]`: count produce dos recursos independientes en el grafo.

```
terraform state show "docker_container.backend[0]"
```

```

mike@miguels-MacBook-Pro terraform % terraform state show "docker_container.backend[0]"
# docker_container.backend[0]:
resource "docker_container" "backend" {
  attach           = false
  bridge           = null
  command          = [
    "npm",
    "start",
  ]
  container_read_refresh_timeout_milliseconds = 15000
  cpu_set          = null
  cpu_shares       = 0
  domainname      = null
  entrypoint       = [
    "docker-entrypoint.sh",
  ]
  env              = (sensitive value)
  hostname         = "backend-1"
  id               = "170d7ff57cddea7a2bb8db75229f14b5f2ecbc3ebb8a252211240fcdc4d79d69"
  image            = "sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8ca"
  init             = false
  ipc_mode         = "private"
  log_driver       = "json-file"
  logs            = false
  max_retry_count  = 0
  memory           = 0
  memory_reservation = 0
  memory_swap      = 0
  must_run         = true
  name             = "workshop-backend-1"
  network_data     = [
    {
      gateway          = "172.21.0.1"
      global_ipv6_address = null
      global_ipv6_prefix_length = 0
      ip_address        = "172.21.0.4"
      ip_prefix_length  = 16
      ipv6_gateway      = null
      mac_address       = "02:d4:fe:31:ec:1b"
      network_name      = "iac-workshop-net"
    },
  ],
  network_mode     = "bridge"
  platform         = "linux"
  privileged        = false
  publish_all_ports = false
  read_only         = false
  remove_volumes   = true
  restart           = "unless-stopped"
  rm               = false
  runtime           = "runc"
  security_opts     = []
  shm_size         = 64
  start            = true
  stdin_open        = false
  stop_signal       = null
  stop_timeout      = 0
  tty              = false
  user             = "node"
  userns_mode       = null
  wait             = false
  wait_timeout      = 60
  working_dir       = "/app"

  networks_advanced {
    aliases = [
      "backend-1",
    ]
    driver_opts = {}
    ipv4_address = null
    ipv6_address = null
    link_local_ips = []
    mac_address = null
    name        = "iac-workshop-net"
  }
}
mike@miguels-MacBook-Pro terraform %

```

Figura 10: Todos los atributos conocidos de una réplica concreta: su ID real de Docker, su IP interna (172.21.0.4), su hostname (backend-1), su alias de red y su usuario no root (node). Las variables de entorno aparecen ocultas por estar marcadas como sensibles.

```
terraform show
```

```

mike@Miguels-MacBook-Pro terraform % terraform show
dockerfile      = "Dockerfile"
extra_hosts     = []
isolation       = null
network_mode    = null
platform        = null
provenance      = null
remote_context  = null
remove          = true
sbom            = null
security_opt    = []
session_id      = null
tag             = []
target          = null
use_legacy_builder = false
version         = null
}
}

# docker_image.mysql:
resource "docker_image" "mysql" {
  id      = "sha256:7dcddc01f13bab2f15cde67d44d01f61fc9f99fe7785e86196dfc07d358ae2b"
  image_id = "sha256:7dcddc01f13bab2f15cde67d44d01f61fc9f99fe7785e86196dfc07d358ae2b"
  keep_locally = true
  name       = "mysql:8.0"
  repo_digest = "mysql@sha256:7dcddc01f13bab2f15cde67d44d01f61fc9f99fe7785e86196dfc07d358ae2b"
}

# docker_image.nginx:
resource "docker_image" "nginx" {
  id      = "sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10"
  image_id = "sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10"
  keep_locally = true
  name       = "nginx:1.27-alpine"
  repo_digest = "nginx@sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10"
}

# docker_network.workshop_net:
resource "docker_network" "workshop_net" {
  attachable = false
  driver     = "bridge"
  id         = "bbd63b681a36238807005ffbe48ed9a6f3ab23f98553437df1e930da206f3ff7"
  ingress    = false
  internal   = false
  ipam_driver = "default"
  ipv6       = false
  name       = "iac-workshop-net"
  options    = {
    "com.docker.network.enable_ipv4" = "true"
    "com.docker.network.enable_ipv6" = "false"
  }
  scope      = "local"

  ipam_config {
    aux_address = {}
    gateway     = "172.21.0.1"
    ip_range    = null
    subnet      = "172.21.0.0/16"
  }
}

# docker_volume.mysql_data:
resource "docker_volume" "mysql_data" {
  driver      = "local"
  id          = "iac-workshop-mysql-data"
  mountpoint  = "/var/lib/docker/volumes/iac-workshop-mysql-data/_data"
  name        = "iac-workshop-mysql-data"
}

Outputs:

api_health_url = "http://localhost:8080/api/health"
app_url        = "http://localhost:8080"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name    = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform %

```

Figura 11: Volcado completo del estado en formato legible: imágenes, red con su subred 172.21.0.0/16, volumen con su punto de montaje real en el host, y los *outputs*.

5.2 Idempotencia

```
terraform plan
```

Sin haber tocado nada, Terraform re-consulta la infraestructura real, compara contra el estado y concluye que no hay nada que hacer. Esto es la idempotencia declarativa en acción.

```

mike@Miguels-MacBook-Pro terraform % terraform plan
docker_image.mysql: Refreshing state... [id=sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_volume.mysql_data: Refreshing state... [id=iac-workshop-mysql-data]
docker_image.nginx: Refreshing state... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_network.workshop_net: Refreshing state... [id=bbd63b681a36238807005ffbe48ed9a6f3ab23f98553437d1e930da206f3ff7]
docker_image.backend: Refreshing state... [id=sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_image.frontend: Refreshing state... [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_container.mysql: Refreshing state... [id=81fde5253f980e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.frontend: Refreshing state... [id=736eecd20721f6f29aec40649a070b9e6195ba35646fd6fb43876854f2304e6d]
docker_container.backend[0]: Refreshing state... [id=170d7ff57cddea7a2bb8db75229f14b5f2ecbc3ebb8a252211240fcd4d79d69]
docker_container.backend[1]: Refreshing state... [id=f9e1e828a2558cba1e36e82bb9fd1c57140f458a5c677bdd95583ac0b8de8d26]
docker_container.nginx: Refreshing state... [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no
differences, so no changes are needed.
mike@Miguels-MacBook-Pro terraform %

```

Figura 12: No changes. Your infrastructure matches the configuration.

5.3 Provocar y detectar *drift*

El *drift* es la desviación entre la infraestructura real y lo que el código declara, típicamente por un cambio manual. Se simuló borrando un contenedor directamente con el CLI de Docker, sin pasar por Terraform:

```

docker rm -f workshop-backend-2
terraform plan

```

Terraform detectó que `docker_container.backend[1]` ya no existe en la realidad aunque el estado dice que debería, y propuso recrearlo.

```

mike@Miguels-MacBook-Pro terraform % docker rm -f workshop-backend-2
workshop-backend-2
mike@Miguels-MacBook-Pro terraform % terraform plan
docker_volume.mysql_data: Refreshing state... [id=iac-workshop-mysql-data]
docker_image.mysql: Refreshing state... [id=sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_image.nginx: Refreshing state... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_network.workshop_net: Refreshing state... [id=bbd63b681a36238807005ffbe48ed9a6f3ab23f98553437d1e930da206f3ff7]
docker_image.backend: Refreshing state... [id=sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_image.frontend: Refreshing state... [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_container.mysql: Refreshing state... [id=81fde5253f980e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.frontend: Refreshing state... [id=736eecd20721f6f29aec40649a070b9e6195ba35646fd6fb43876854f2304e6d]
docker_container.backend[0]: Refreshing state... [id=170d7ff57cddea7a2bb8db75229f14b5f2ecbc3ebb8a252211240fcd4d79d69]
docker_container.backend[1]: Refreshing state... [id=f9e1e828a2558cba1e36e82bb9fd1c57140f458a5c677bdd95583ac0b8de8d26]
docker_container.nginx: Refreshing state... [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# docker_container.backend[1] will be created
+ resource "docker_container" "backend" {
  + attach              = false
  + bridge              = (known after apply)
  + command             = (known after apply)
  + container_logs      = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + entrypoint          = (known after apply)
  + env                = (sensitive value)
  + exit_code           = (known after apply)
  + hostname            = "backend-2"
  + id                 = (known after apply)
  + image               = "sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8ca"
  + init               = (known after apply)
  + ipc_mode            = (known after apply)
  + log_driver          = (known after apply)
  + logs               = false
  + memory_reservation = 0
  + must_run            = true
  + name               = "workshop-backend-2"
  + network_data        = (known after apply)
  + network_mode        = "bridge"
  + platform            = (known after apply)
  + read_only           = false
  + remove_volumes     = true
  + restart             = "unless-stopped"
  + rm                 = false
  + runtime             = (known after apply)
  + security_opts       = (known after apply)
  + shm_size            = (known after apply)
  + start              = true
  + stdin_open          = false
  + stop_signal         = (known after apply)
  + stop_timeout        = (known after apply)
  + tty                = false
  + wait               = false
  + wait_timeout        = 60
}

```

Figura 13: Detección del *drift*: el recurso desaparecido se marca para creación, con `hostname = "backend-2"` y su alias de red.

```
+ networks_advanced {
+   aliases = [
+     "backend-2",
+   ]
+   driver_opts = []
+   link_local_ips = []
+   name = "iac-workshop-net"
+   # (3 unchanged attributes hidden)
+ }
}
```

Plan: 1 to add, 0 to change, 0 to destroy.

Note: You didn't use the `-out` option to save this plan, so Terraform can't guarantee to take exactly these actions if you run "terraform apply" now.

○ mike@Miguels-MacBook-Pro terraform %

Figura 14: Resumen: 1 to add, 0 to change, 0 to destroy. Terraform propone tocar solo el recurso afectado.

5.4 Corregir el *drift*

```
terraform apply
```

```

mike@Miguels-MacBook-Pro terraform % terraform apply

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create

Terraform will perform the following actions:

# docker_container.backend[1] will be created
+ resource "docker_container" "backend" {
  + attach                = false
  + bridge                = (known after apply)
  + command               = (known after apply)
  + container_logs        = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  + entrypoint            = (known after apply)
  + env                   = (sensitive value)
  + exit_code             = (known after apply)
  + hostname              = "backend-2"
  + id                    = (known after apply)
  + image                  = "sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8ca"
  + init                  = (known after apply)
  + ipc_mode              = (known after apply)
  + log_driver             = (known after apply)
  + logs                  = false
  + memory_reservation     = 0
  + must_run              = true
  + name                  = "workshop-backend-2"
  + network_data          = (known after apply)
  + network_mode          = "bridge"
  + platform              = (known after apply)
  + read_only             = false
  + remove_volumes        = true
  + restart               = "unless-stopped"
  + rm                    = false
  + runtime                = (known after apply)
  + security_opts          = (known after apply)
  + shm_size              = (known after apply)
  + start                 = true
  + stdin_open            = false
  + stop_signal            = (known after apply)
  + stop_timeout          = (known after apply)
  + tty                   = false
  + wait                  = false
  + wait_timeout          = 60

  + healthcheck (known after apply)

  + labels (known after apply)

  + networks_advanced {
    + aliases = [
      + "backend-2",
    ]
    + driver_opts = []
    + link_local_ips = []
    + name = "iac-workshop-net"
    # (3 unchanged attributes hidden)
  }
}

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
Terraform will perform the actions described above.
Only 'yes' will be accepted to approve.

Enter a value: yes

docker_container.backend[1]: Creating...
docker_container.backend[1]: Creation complete after 1s [id=24763f2d397af1e17198ed18d4c55d9cac5b0dadd67ae6c49b58d29f3119c62b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

api_health_url = "http://localhost:8080/api/health"
app_url = "http://localhost:8080"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform % █

```

Figura 15: El apply recrea únicamente el contenedor faltante.

```

Plan: 1 to add, 0 to change, 0 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_container.backend[1]: Creating...
docker_container.backend[1]: Creation complete after 1s [id=24763f2d397af1e17198ed18d4c55d9cac5b0dadd67ae6c49b58d29f3119c62b]

Apply complete! Resources: 1 added, 0 changed, 0 destroyed.

Outputs:

api_health_url = "http://localhost:8080/api/health"
app_url = "http://localhost:8080"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform %

```

Figura 16: 1 added, 0 changed, 0 destroyed: el resto del estado seguía coincidiendo con la realidad y no se tocó.

5.5 Reemplazar un recurso puntual

Cuando no hay *drift* pero se quiere forzar una instancia nueva y limpia de un recurso concreto:

```
terraform apply -replace="docker_container.backend[0]"
```

```

mike@Miguels-MacBook-Pro terraform % terraform apply -replace="docker_container.backend[0]"
docker_volume.mysql_data: Refreshing state... [id=iac-workshop-mysql-data]
docker_image.mysql: Refreshing state... [id=sha256:7dcdc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_image.nginx: Refreshing state... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_network.workshop_net: Refreshing state... [id=bbd63b681a36238807005ffbe48ed9a6f3ab23f98553437dfe930da206f3ff7]
docker_image.backend: Refreshing state... [id=sha256:8a48594775c4f5dd0bf8db300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_image.frontend: Refreshing state... [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_container.frontend: Refreshing state... [id=736eecd20721f6f29aec40649a070b9e6195ba35646fd6fb43876854f2304e6d]
docker_container.mysql: Refreshing state... [id=81fde5253f988e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.backend[0]: Refreshing state... [id=170d7ff57cddea7a2bb8db75229f14b5f2ecbc3ebb8a252211240fcdc4d79d69]
docker_container.backend[1]: Refreshing state... [id=24763f2d397af1e17198ed18d4c55d9cac5b0dadd67ae6c49b58d29f3119c62b]
docker_container.nginx: Refreshing state... [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]

Terraform used the selected providers to generate the following execution plan. Resource actions are indicated with the following symbols:
-/+ destroy and then create replacement

Terraform will perform the following actions:

# docker_container.backend[0] will be replaced, as requested
-/+ resource "docker_container" "backend" {
  + bridge           = (known after apply)
  ~ command          = [
    - "npm",
    - "start",
  ] -> (known after apply)
  + container_logs    = (known after apply)
  - cpu_shares        = 0 -> null
  - dns               = [] -> null
}

```

Figura 17: El plan marca el recurso como replaced, as requested y usa el símbolo -/+ (destruir y luego crear).


```

Plan: 1 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_container.backend[0]: Destroying... [id=170d7ff57cddea7a2bb8db75229f14b5f2ecbc3ebb8a252211240fcdc4d79d69]
docker_container.backend[0]: Destruction complete after 1s
docker_container.backend[0]: Creating...
docker_container.backend[0]: Creation complete after 0s [id=c8321f938ba22bdbbc27341b0c460dfdc68e44ef51e5cf2e1a978b0430be4d1]

Apply complete! Resources: 1 added, 0 changed, 1 destroyed.

Outputs:
api_health_url = "http://localhost:8080/api/health"
app_url = "http://localhost:8080"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform %

```

Figura 18: 1 added, 0 changed, 1 destroyed: solo backend-1 se recreó; backend-2 quedó intacto.

Esta bandera sustituye al antiguo `terraform taint`, desaconsejado desde Terraform 0.15.2.

5.6 Renombrar una dirección sin destruir el recurso

Si se renombra un recurso solo en el código, Terraform planea destruir el antiguo y crear uno nuevo, aunque en la práctica sea el mismo contenedor. `state mv` evita esa recreación innecesaria:

```
terraform state mv docker_container.frontend docker_container.web
```

```

mike@Miguels-MacBook-Pro terraform % terraform state mv docker_container.frontend docker_container.web
Move "docker_container.frontend" to "docker_container.web"
Successfully moved 1 object(s).
mike@Miguels-MacBook-Pro terraform %

```

Figura 19: La dirección se mueve dentro del estado: Successfully moved 1 object(s).

```

mysql_volume_name = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform % terraform state mv docker_container.frontend docker_container.web
Move "docker_container.frontend" to "docker_container.web"
Successfully moved 1 object(s).
mike@Miguels-MacBook-Pro terraform % terraform state list
docker_container.backend[0]
docker_container.backend[1]
docker_container.mysql
docker_container.nginx
docker_container.web
docker_image.backend
docker_image.frontend
docker_image.mysql
docker_image.nginx
docker_network.workshop_net
docker_volume.mysql_data
mike@Miguels-MacBook-Pro terraform %

```

Figura 20: El estado ya contiene `docker_container.web` en lugar de `docker_container.frontend`, sin que el contenedor real se haya tocado.

En este punto el estado y el código quedan momentáneamente desincronizados, y el siguiente `plan` falla. Es un error esperado y didáctico: `nginx.tf` seguía apuntando a la dirección vieja.

```

mike@Miguels-MacBook-Pro terraform % terraform plan

Error: Reference to undeclared resource

  on nginx.tf line 35, in resource "docker_container" "nginx":
  35:   depends_on = [docker_container.backend, docker_container.frontend]

A managed resource "docker_container" "frontend" has not been declared in the root module.

mike@Miguels-MacBook-Pro terraform % █

```

Figura 21: Reference to undeclared resource: nginx.tf línea 35 todavía referencia docker_container.frontend.

Tras renombrar también el bloque en el código, el plan vuelve a mostrar “sin cambios” en lugar de un *destroy/create*, que es exactamente el objetivo del ejercicio.

```

mike@Miguels-MacBook-Pro terraform % terraform plan

Error: Reference to undeclared resource

  on nginx.tf line 35, in resource "docker_container" "nginx":
  35:   depends_on = [docker_container.backend, docker_container.frontend]

A managed resource "docker_container" "frontend" has not been declared in the root module.

mike@Miguels-MacBook-Pro terraform % terraform plan
docker_image.mysql: Refreshing state... [id=sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_network.workshop_net: Refreshing state... [id=bbd63b681a36238807005ffbe48ed9a6f3ab23f98553437df1e930da206f3ff7]
docker_volume.mysql_data: Refreshing state... [id=iac-workshop-mysql-data]
docker_image.nginx: Refreshing state... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_image.backend: Refreshing state... [id=sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_image.frontend: Refreshing state... [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_container.web: Refreshing state... [id=736eecd20721f6f29aec40649a070b9e6195ba35646fd6fb43876854f2304e6d]
docker_container.mysql: Refreshing state... [id=81fde5253f988e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.backend[1]: Refreshing state... [id=24763f2d397af1e17198ed18d4c55d9cac5b0dadd67a6c49b58d29f3119c62b]
docker_container.backend[0]: Refreshing state... [id=c8321f938ba22b0bbcc27341b0c460dfdc68e44ef51e5cf2e1a978b0430be4d1]
docker_container.nginx: Refreshing state... [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]

No changes. Your infrastructure matches the configuration.

Terraform has compared your real infrastructure against your configuration and found no
differences, so no changes are needed.
mike@Miguels-MacBook-Pro terraform % █

```

Figura 22: Con código y estado alineados, Terraform no propone ninguna acción.

5.7 Importar un recurso creado manualmente

Este es probablemente el ejercicio más ilustrativo. Se creó un contenedor de Redis **completamente fuera de Terraform**, como si fuera un recurso heredado:

```
docker network connect iac-workshop-net --alias redis-manual \
  $(docker run -d --name manual-redis redis:7-alpine)
```

```

mike@Miguels-MacBook-Pro terraform %

docker network connect iac-workshop-net --alias redis-manual $(docker run -d --name manual-redis redis:7-alpine)
Unable to find image 'redis:7-alpine' locally
7-alpine: Pulling from library/redis
2ead3bb52b36: Pull complete
4f4fb700ef54: Pull complete
0959c9ac2be7: Pull complete
1720d483913a: Pull complete
9ca61e94b8b4: Pull complete
f771ad69e7ef: Pull complete
2dd7199cff98: Pull complete
b54311e1c98c: Download complete
782d666cd29f: Download complete
Digest: sha256:ff02b58f971e7d7d156a1267e283fcbbeee91773b6aa36c49dac28ecfe28eadf
Status: Downloaded newer image for redis:7-alpine
mike@Miguels-MacBook-Pro terraform % █

```

Figura 23: Docker descarga redis:7-alpine y crea el contenedor manual-redis al margen de Terraform.

Después se escribió en `terraform/redis.tf` el bloque que *debería* describir ese contenedor, y se adoptó en el estado sin recrearlo:

```
terraform import docker_container.redis \
  ${docker inspect -f '{{.Id}}' manual-redis}
```

```
● mike@Miguels-MacBook-Pro terraform % terraform import docker_container.redis $(docker inspect -f '{{.Id}}' manual-redis)
docker_container.redis: Importing from ID "5f4d4039bb11e4726dc82ddddeef5538237bb200e3783a45dc340bc445f08ee39"...
docker_container.redis: Import prepared!
Prepared docker_container for import
docker_container.redis: Refreshing state... [id=5f4d4039bb11e4726dc82ddddeef5538237bb200e3783a45dc340bc445f08ee39]

Import successful!

The resources that were imported are shown above. These resources are now in
your Terraform state and will henceforth be managed by Terraform.

● mike@Miguels-MacBook-Pro terraform %
```

Figura 24: Import successful! El contenedor pasa a estar administrado por Terraform sin haber sido destruido ni vuelto a crear.

```
● mike@Miguels-MacBook-Pro terraform % terraform plan
docker_volume.mysql_data: Refreshing state... [id=iac-workshop-mysql-data]
docker_network.workshop_net: Refreshing state... [id=bdd63b681a36238807005ffbe48ed9a6f3ab23f98553437df1e930da206f3ff7]
docker_image.nginx: Refreshing state... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_image.mysql: Refreshing state... [id=sha256:7dcddc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_image.frontend: Refreshing state... [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_image.backend: Refreshing state... [id=sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_container.frontend: Refreshing state... [id=736eecd20721f6f29aec40649a070b9e6195ba35646fd6fb43876854f2304e6d]
docker_container.redis: Refreshing state... [id=5f4d4039bb11e4726dc82ddddeef5538237bb200e3783a45dc340bc445f08ee39]
docker_container.mysql: Refreshing state... [id=81fde5253f988e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.backend[1]: Refreshing state... [id=24763f2d397af1e17198ed18d4c55d9cac5b0dadd67ae6c49b58d29f3119c62b]
docker_container.backend[0]: Refreshing state... [id=c8321f938ba22bdbc27341b0c460dfdc68e44ef51e5cf2e1a978b0430be4d1]
docker_container.nginx: Refreshing state... [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]

Terraform used the selected providers to generate the following execution plan. Resource
actions are indicated with the following symbols:
+ create
-/+ destroy and then create replacement

Terraform will perform the following actions:

# docker_container.redis must be replaced
-/+ resource "docker_container" "redis" {
  + attach              = false
  + bridge              = (known after apply)
  + command             = [
    - "redis-server",
    ] -> (known after apply)
  + container_logs      = (known after apply)
  + container_read_refresh_timeout_milliseconds = 15000
  - cpu_shares          = 0 -> null
  - dns                 = [] -> null
  - dns_opts            = [] -> null
  - dns_search          = [] -> null
  - network_mode        = [] -> null
}
```

Figura 25: El plan posterior muestra `docker_container.redis must be replaced`: algunos atributos del código no coinciden exactamente con los del contenedor importado (por ejemplo el `command`), lo que fuerza una recreación. Es un resultado normal del ejercicio.

```
# docker_image.redis will be created
+ resource "docker_image" "redis" {
+   id           = (known after apply)
+   image_id     = (known after apply)
+   name        = "redis:7-alpine"
+   repo_digest = (known after apply)
+ }

Plan: 2 to add, 0 to change, 1 to destroy.

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take
exactly these actions if you run "terraform apply" now.
mike@Miguels-MacBook-Pro terraform %
```

Figura 26: 2 to add, 0 to change, 1 to destroy: se crea además `docker_image.redis`, porque el recurso de imagen no soporta `import` en este proveedor. Como la imagen ya existía localmente, es un efecto inofensivo.

```
Plan: 2 to add, 0 to change, 1 to destroy.

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_container.redis: Destroying... [id=5f4d4039bb11e4726dc82dddeef5538237bb200e3783a45dc340bc445f08ee39]
docker_container.redis: Destruction complete after 0s
docker_image.redis: Creating...
docker_image.redis: Creation complete after 0s [id=sha256:ff02b58f971e7d7d156a1267e283fcbbeee91773b6aa36c49dac28ecfe28eadfredis:7-alpine]
docker_container.redis: Creating...
docker_container.redis: Creation complete after 1s [id=454056167f4617c7eb2b75196c662189fcac7365a733a850fdde056c31e5e32b]

Apply complete! Resources: 2 added, 0 changed, 1 destroyed.

Outputs:
api_health_url = "http://localhost:8080/api/health"
app_url        = "http://localhost:8080"
backend_container_names = [
  "workshop-backend-1",
  "workshop-backend-2",
]
mysql_container_name = "workshop-mysql"
mysql_volume_name   = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform %
```

Figura 27: El `apply` completa la adopción del recurso.

Docker Compose no tiene ningún comando equivalente a `terraform import`: solo puede gestionar lo que él mismo creó. El ejercicio deja además una lección extra: no todos los tipos de recurso de un proveedor soportan importación, y conviene revisar la documentación antes de asumirlo.

5.8 Limpieza del ejercicio y una observación

```
rm terraform/redis.tf
docker rm -f manual-redis
terraform state rm docker_container.redis docker_image.redis
```

```

● mike@Miguels-MacBook-Pro terraform % rm terraform/redis.tf
docker rm -f manual-redis
terraform state rm docker_container.redis docker_image.redis
rm: terraform/redis.tf: No such file or directory
manual-redis
Removed docker_container.redis
Removed docker_image.redis
Successfully removed 2 resource instance(s).
○ mike@Miguels-MacBook-Pro terraform % █

```

Figura 28: `terraform state rm` saca los dos recursos del estado sin destruirlos en la realidad. Nótese, sin embargo, el mensaje `rm: terraform/redis.tf: No such file or directory`.

Merece la pena detenerse en ese detalle. El comando se ejecutó desde dentro de `terraform/`, así que la ruta relativa `terraform/redis.tf` no resolvía y el fichero **no se borró**. El resultado fue que, en el siguiente `apply` (durante el escalado de la sección siguiente), Terraform volvió a crear Redis a partir de ese `redis.tf` superviviente.

Lejos de ser solo un descuido, es una demostración práctica del principio de fondo del taller: **el código es la fuente de verdad**. Sacar algo del estado no lo elimina de la configuración, y mientras el bloque siga declarado Terraform lo reconstruirá. La ruta correcta desde `terraform/` habría sido simplemente `rm redis.tf`.

5.9 Resumen de la comparación con Docker Compose

Tabla 3: Qué puede hacer cada herramienta

Capacidad	Compose	Terraform
Detectar si la configuración cambió	Sí	Sí
Detectar <i>drift</i> por cambios manuales	No	Sí (<code>plan</code>)
Vista previa con <i>diff</i> explícito	No	Sí (<code>plan</code>)
Consultar atributos completos de un recurso	Limitado	Sí (<code>state show</code>)
Renombrar sin recrear el recurso real	No	Sí (<code>state mv</code>)
Adoptar un recurso creado por fuera	No	Sí (<code>import</code>)
Reemplazar un recurso puntual bajo demanda	No	Sí (<code>-replace</code>)
Bloqueo contra aplicaciones concurrentes	No	Sí (<i>backend</i> remoto)

6. Escalar el backend

Basta cambiar una sola variable en `terraform.tfvars`:

```
backend_replica_count = 4
```

```
terraform plan
```

```

Plan: 5 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ backend_container_names = [
    # (1 unchanged element hidden)
    "workshop-backend-2",
    + "workshop-backend-3",
    + "workshop-backend-4",
  ]

Note: You didn't use the -out option to save this plan, so Terraform can't guarantee to take exactly
these actions if you run "terraform apply" now.
○ mike@Miguels-MacBook-Pro terraform %

```

Figura 29: El plan añade workshop-backend-3 y workshop-backend-4 a la lista de *outputs*.

Lo interesante no son las réplicas nuevas, sino que Nginx también aparece marcado para recreación. Su configuración se genera con `templatefile()` a partir de la lista de réplicas; al cambiar esa lista cambia el contenido del fichero, y como el fichero se sube al crear el contenedor, no se puede parchear en caliente. Es el principio de infraestructura inmutable, visible en la práctica.

```

# docker_container.nginx must be replaced
-/+ resource "docker_container" "nginx" {
  + bridge                = (known after apply)
  ~ command               = [
    - "nginx",
    - "-g",
    - "daemon off;",
  ] -> (known after apply)
  + container_logs        = (known after apply)
  - cpu_shares             = 0 -> null
  - dns                   = [] -> null
  - dns_opts              = [] -> null

```

Figura 30: `docker_container.nginx must be replaced`: la recreación no viene de un cambio manual, sino de una dependencia sobre datos que cambiaron.

```
terraform apply
```

```

Plan: 5 to add, 0 to change, 1 to destroy.

Changes to Outputs:
  ~ backend_container_names = [
    # (1 unchanged element hidden)
    "workshop-backend-2",
    + "workshop-backend-3",
    + "workshop-backend-4",
  ]

Do you want to perform these actions?
  Terraform will perform the actions described above.
  Only 'yes' will be accepted to approve.

  Enter a value: yes

docker_image.redis: Creating...
docker_container.nginx: Destroying... [id=e628e258f1d76e17b4043283fffc3338ce698ac75c0982b3457c4d3bc49ed76a]
docker_image.redis: Creation complete after 1s [id=sha256:ff02b58f971e7d7d156a1267e283fcbbeee91773b6aa36c49dac28ecfe28eadfredis:7-alpine]
docker_container.redis: Creating...
docker_container.nginx: Destruction complete after 1s
docker_container.backend[3]: Creating...
docker_container.backend[2]: Creating...
docker_container.redis: Creation complete after 0s [id=ec125016ad515b6ab95758d887cde233a39a9f1e2bfff61d68dbeb1f21a9f4994]
docker_container.backend[2]: Creation complete after 1s [id=2e01b2d1e1c636452884f5e5d5cfdcb6f143ba1f7024ba90e456cac83ec8f0ee]
docker_container.backend[3]: Creation complete after 1s [id=60f26a08c8c9e0f37d6488cdb3a07634f222838382e6245df8df39c6b2f05267]
docker_container.nginx: Creating...
docker_container.nginx: Creation complete after 0s [id=69712b9f806394c81b656689e45ecba1d8576ed4e094a18733d395d78e23b733]

Apply complete! Resources: 5 added, 0 changed, 1 destroyed.

Outputs:
  api_health Follow link \(cmd + click\) :8080/api/health"
  app_url = "http://localhost:8080"
  backend_container_names = [
    "workshop-backend-1",
    "workshop-backend-2",
    "workshop-backend-3",
    "workshop-backend-4",
  ]
  mysql_container_name = "workshop-mysql"
  mysql_volume_name = "iac-workshop-mysql-data"
mike@Miguels-MacBook-Pro terraform %

```

Figura 31: 5 added, 0 changed, 1 destroyed: dos backends nuevos, Nginx destruido y recreado, y Redis recreado a partir del `redis.tf` que había sobrevivido a la limpieza.

Volviendo al navegador, el botón de balanceo ahora reparte las peticiones entre cuatro hostnames distintos.

Taller IaC · Terraform + Docker

React → Nginx (balanceador) → Node.js (réplicas) → MySQL, todo desplegado con Terraform.

Conexión a la base de datos

Total de visitas registradas en MySQL: 2
Petición atendida por: backend-1

Balanceo de carga entre réplicas

Enviar 6 peticiones a /api/health

- Petición 1 → atendida por backend-2
- Petición 2 → atendida por backend-3
- Petición 3 → atendida por backend-4
- Petición 4 → atendida por backend-1
- Petición 5 → atendida por backend-2
- Petición 6 → atendida por backend-3

Figura 32: La misma interfaz, ahora repartiendo entre backend-1 a backend-4. El contador de visitas subió a 2, leído desde el volumen persistente de MySQL.

7. Destruir el entorno

```
terraform destroy
```



```
mike@Miguels-MacBook-Pro terraform % terraform destroy

- ipam_config {
  - aux_address = {} -> null
  - gateway     = "172.21.0.1" -> null
  - subnet     = "172.21.0.0/16" -> null
  # (1 unchanged attribute hidden)
}

# docker_volume.mysql_data will be destroyed
- resource "docker_volume" "mysql_data" {
  - driver       = "local" -> null
  - driver_opts = {} -> null
  - id          = "iac-workshop-mysql-data" -> null
  - mountpoint  = "/var/lib/docker/volumes/iac-workshop-mysql-data/_data" -> null
  - name        = "iac-workshop-mysql-data" -> null
}

Plan: 0 to add, 0 to change, 15 to destroy.

Changes to Outputs:
- api_health_url      = "http://localhost:8080/api/health" -> null
- app_url             = "http://localhost:8080" -> null
- backend_container_names = [
  - "workshop-backend-1",
  - "workshop-backend-2",
  - "workshop-backend-3",
  - "workshop-backend-4",
] -> null
- mysql_container_name = "workshop-mysql" -> null
- mysql_volume_name    = "iac-workshop-mysql-data" -> null

Do you really want to destroy all resources?
Terraform will destroy all your managed infrastructure, as shown above.
There is no undo. Only 'yes' will be accepted to confirm.

Enter a value: yes

docker_container.redis: Destroying... [id=ec125016ad515b6ab95758d887cde233a39a9f1e2bfff61d68dbef1f21a9f4994]
docker_container.nginx: Destroying... [id=69712b9f806394c81b656689e45ecba1d8576ed4e094a18733d395d78e23b733]
docker_container.redis: Destruction complete after 0s
docker_image.redis: Destroying... [id=sha256:ff02b58f971e7d7d156a1267e283fcbbee91773b6aa36c49dac28ecfe28eadfredis:7-alpine]
docker_container.nginx: Destruction complete after 0s
docker_image.nginx: Destroying... [id=sha256:65645c7bb6a0661892a8b03b89d0743208a18dd2f3f17a54ef4b76fb8e2f2a10nginx:1.27-alpine]
docker_container.backend[2]: Destroying... [id=2e01b2d1e1c636452884f5e5d5cfdcb6f143balf7024ba90e456cac83ec8f0ee]
docker_image.nginx: Destruction complete after 0s
docker_container.backend[0]: Destroying... [id=c8321f938ba22bdbc27341b0c460dfdcd68e44ef51e5cf2e1a978b0430be4d1]
docker_container.frontend: Destroying... [id=736eecd20721f6f29aec40649a070b9a6195ba35646fd6fb43876854f2304e6d]
docker_container.backend[1]: Destroying... [id=24763f2d397af1e17198ed1844c55d9cac5b0dadd67ae6c49b58d29f3119c62b]
docker_container.backend[3]: Destroying... [id=60f26a08c8c9e0f37d6488cdb3a07634f222838382e6245df8df39c6b2f05267]
docker_container.backend[2]: Destruction complete after 1s
docker_container.backend[3]: Destruction complete after 1s
docker_container.backend[1]: Destruction complete after 1s
docker_container.backend[0]: Destruction complete after 1s
docker_image.backend: Destroying... [id=sha256:8a48594775c4f5dd0bfbf8dba300a3bac14417ba549a924867f49a6a90e8f8caiac-workshop-backend:local]
docker_container.mysql: Destroying... [id=81fde5253f988e51fecb0814a065b79d5f2a693ad31c8612955b8726987c468d]
docker_container.frontend: Destruction complete after 1s
docker_image.frontend: Destroying... [id=sha256:717da3a9b8ef82d66440ee4186a3f808bbdd028bb568fab7ac385ca42702b5f3iac-workshop-frontend:local]
docker_image.redis: Destruction complete after 1s
docker_container.mysql: Destruction complete after 1s
docker_image.frontend: Destruction complete after 1s
docker_volume.mysql_data: Destroying... [id=iac-workshop-mysql-data]
docker_network.workshop_net: Destroying... [id=bdd63b681a36238807005ffbe48ed9a6f3ab23f98553437df1e930da206f3ff7]
docker_image.mysql: Destroying... [id=sha256:7dcdc01f13bab2f15cde676d44d01f61fc9f99fe7785e86196dfc07d358ae2bmysql:8.0]
docker_image.mysql: Destruction complete after 0s
docker_volume.mysql_data: Destruction complete after 2s
docker_network.workshop_net: Destruction complete after 3s

Destroy complete! Resources: 15 destroyed.
mike@Miguels-MacBook-Pro terraform %
```

Figura 33: Destroy complete! Resources: 15 destroyed. Terraform elimina todo en orden inverso de dependencias: primero los contenedores, luego las imágenes, y al final el volumen y la red.

Los 15 recursos son los 11 originales más los 2 backends añadidos al escalar y los 2 de Redis. Conviene subrayar que `destroy` se llevó también el volumen `iac-workshop-mysql-data` y, con él, los datos de MySQL. En un entorno con datos reales, `lifecycle { prevent_destroy = true }` sobre ese volumen es la red de seguridad contra exactamente este comando.

8. Conclusiones

- El flujo `init` → `plan` → `apply` se completó sin incidencias: 11 recursos creados y la aplicación respondiendo en `localhost:8080`, con el balanceo entre réplicas verificable desde el navegador.
- El archivo de estado es la diferencia real frente a Docker Compose. Detectar el borrado manual de un contenedor, renombrar una dirección sin recrear nada y adoptar un contenedor creado a mano son operaciones que Compose sencillamente no puede ofrecer, porque no mantiene un registro consultable de la infraestructura.

- El escalado dejó ver el efecto en cascada más interesante del taller: cambiar una variable no solo añadió réplicas, sino que obligó a recrear Nginx, porque su configuración se deriva de esa misma variable. Terraform razona sobre el grafo de dependencias, no sobre ficheros sueltos.
- El descuido en la limpieza de Redis resultó ser el ejemplo más claro de todo el ejercicio: sacar un recurso del estado no lo borra del código, y mientras el bloque siga declarado Terraform lo reconstruirá en el siguiente **apply**.
- El **destroy** final eliminó los 15 recursos, incluido el volumen de MySQL, confirmando por qué **prevent_destroy** es necesario en cuanto hay datos que importan.